

SQL (Structured Query Language)

Clause's in SQL :-

1) GROUP BY CLAUSE

- The GROUP BY clause in SQL is used to arrange identical data into groups.
- It is often used with aggregate functions like COUNT(), SUM(), AVG(), MIN(), and MAX() to perform calculations on each group.
- The grouping is based on one or more columns specified in the GROUP BY clause.

Syntax :-

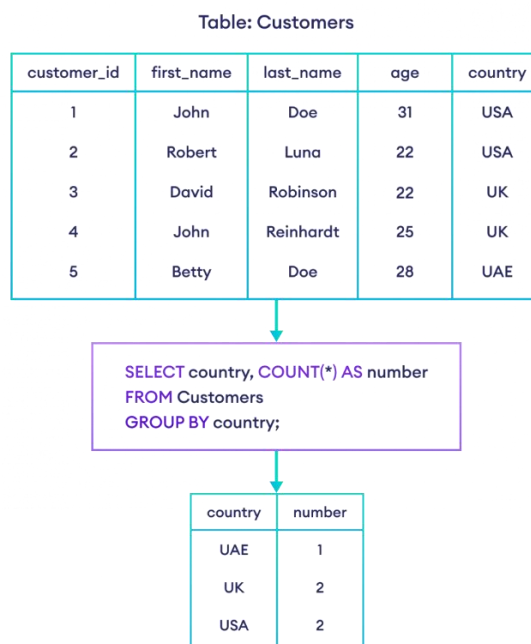
```
SELECT column1, column2, ...  
FROM table  
GROUP BY columnA, columnB, ...;
```

Here,

- column1, column2 ... are the columns of the table
- table is the name of the table
- columnA, columnB ... are the column(s) based on which the rows will be grouped

Example :-

```
SELECT country, COUNT(*) AS number  
FROM Customers  
GROUP BY country;
```



2) HAVING CLAUSE

- The HAVING clause is used to filter records after the GROUP BY clause is applied.
- It is used to filter data based on aggregate functions like SUM(), AVG(), COUNT(), MAX(), MIN().
- The WHERE clause cannot be used with aggregate functions, but HAVING can.
- WHERE filters rows before grouping, while HAVING filters groups after grouping.

Syntax :-

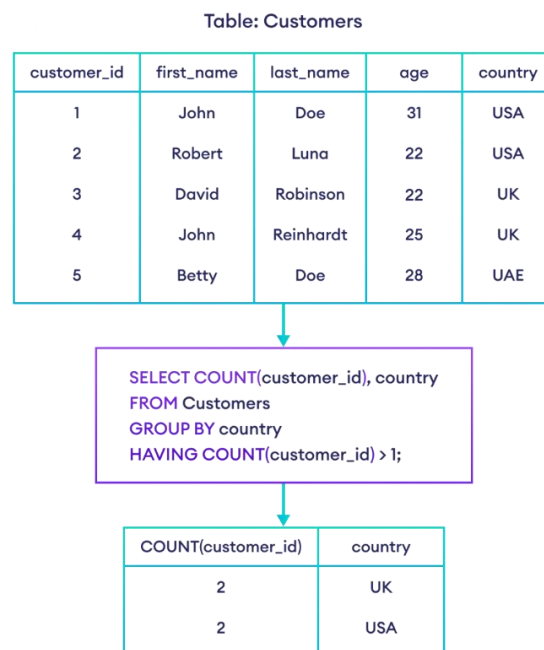
```
SELECT AggFunc(column), extra_columns  
FROM table  
GROUP BY target_column  
HAVING condition
```

Here,

- AggFunc(column) refers to any aggregate function applied to a column
- extra_columns are other extra columns to filter
- GROUP BY groups the data by target_column
- HAVING condition compares the column to certain conditions that require filtering

Example :-

```
SELECT COUNT(customer_id), country  
FROM Customers  
GROUP BY country  
HAVING COUNT(customer_id) > 1;
```



LIKE in SQL :-

- The LIKE operator in SQL is used to search for a specified pattern in a column.
- It is mostly used in the WHERE clause to filter rows based on pattern matching.
- LIKE works with wildcard characters to match part of a string.
- Two main wildcards are:
 1. % → Represents zero, one, or multiple characters.
 2. _ → Represents exactly one character.
- The LIKE operator is case-insensitive in some databases (e.g., MySQL) but case-sensitive in others (e.g., PostgreSQL depending on collation).

Syntax :-

```
SELECT column_name  
FROM table_name  
WHERE column_name LIKE pattern;
```

1) Example with %:

```
SELECT * FROM employees  
WHERE name LIKE 'D%'; -- names starting with D
```

2) Example with _:

```
SELECT * FROM employees  
WHERE name LIKE '_o%'; -- names where 2nd letter is 'o'
```

Table: Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

SELECT *
FROM Customers
WHERE country **LIKE** 'UK';

customer_id	first_name	last_name	age	country
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK